Regression and Practical Machine Learning

A Complete Beginner-to-Intermediate Guide with Python, NumPy, Pandas, Matplotlib, and Scikit-learn

Contents

Part 1 — Machine Learning and Regression Foundations

1.1 What Is Machine Learning?

Machine learning is the practice of writing programs that improve at a task by looking at data, instead of being told explicit step-by-step rules. A traditional program says "if X, then do Y" using rules a human wrote by hand. A machine-learning program instead says "here are many examples of inputs and correct outputs — find a pattern that connects them," and the resulting pattern (the **model**) is then used to make decisions about data it has never seen.

Why do we need this? Many real-world relationships (what makes a house expensive, what makes an email spam) are too complex, too subtle, or too numerous to describe in hand-written rules. Machine learning lets the data itself reveal the relationship.

1.2 Supervised vs. Unsupervised Learning

- **Supervised learning**: The training data includes both the inputs and the correct answers (labels). The model learns a mapping from inputs to known outputs. Regression and classification both belong here.
- **Unsupervised learning**: The training data has no labels. The model looks for structure on its own (e.g., grouping similar customers together).

Regression is always supervised: for every historical example, we already know the true numerical outcome (e.g., the house's actual sale price), and we use that truth to teach the model.

1.3 Feature, Target, and Sample — the Vocabulary of a Dataset

- **Feature**: An input variable used to make a prediction (e.g., square footage, number of bedrooms). Also called a predictor or independent variable.
- **Target / label**: The value we are trying to predict (e.g., sale price). Also called the dependent variable.
- **Sample / observation**: One row of data — one complete real-world instance, with its features and (during training) its known target.

Think of a spreadsheet: each **row** is a sample, each **column** (except the target column) is a feature, and the target column holds the answer key.

1.4 Numerical vs. Categorical Data

- **Numerical data**: Expressed as numbers where arithmetic is meaningful (age, price, temperature).
- **Categorical data**: Represents a label or group with no inherent arithmetic meaning (city name, color, yes/no).

This distinction matters because regression models operate on numbers. Categorical columns must eventually be converted into numeric form (Part 24) before a regression model can use them.

1.5 Continuous vs. Discrete Variables

- **Continuous**: Can take any value within a range, including fractions (price, temperature, time).
- **Discrete**: Restricted to specific, usually whole-number, values (number of bedrooms, number of children).

Regression is built around predicting continuous numerical targets, though it can also be applied to discrete numerical targets in practice.

1.6 Training Data vs. Testing Data

- **Training data**: The portion of the dataset the model is allowed to learn from.
- **Testing data**: A portion held back and never shown to the model during training, used afterward to check how well the model generalizes to new, unseen data.

This split exists because a model that has memorized its training data can look perfect on paper while being useless in practice. Testing data simulates "the real world the model hasn't seen yet." This idea is expanded fully in Part 14.

1.7 What Does "Learning From Data" Actually Mean?

Concretely, learning means: the model has internal numbers (**parameters**) that start at some default, and an algorithm systematically adjusts those numbers so that the model's predictions get closer and closer to the true answers in the training data. "Training a model" and "finding the parameter values that minimize prediction error" mean the same thing. This idea is the mathematical core of everything that follows.

1.8 What Is Regression?

Regression is a supervised-learning technique used to predict a **continuous numerical value** from one or more input features.

- Regression is supervised because every training example already has a known correct numeric answer, and the model is scored against that answer while it learns.
- The output of a regression model is always a number that can, in principle, take on a continuous range of values (₹75,00,000; 42.7°C; 3.2 days), not a category.

Real-world examples of regression problems

Problem	Features (inputs)	Target (output)
House-price prediction	area, location, bedrooms	price (₹)
Salary prediction	experience, education, role	salary (₹/year)
Sales forecasting	past sales, season, promotions	units sold
Demand prediction	price, competitor price, weather	demand (units)
Delivery-time prediction	distance, traffic, time of day	delivery time (minutes)
Temperature prediction	humidity, pressure, season	temperature (°C)
Customer spending prediction	past purchases, income, age	amount spent (₹)

1.9 Regression vs. Classification

	Regression	Classification
Output type	Continuous number	Category / class label
Example question	"How much will this house sell for?"	"Is this email spam or not spam?"
Example output	₹75,00,000	"Spam"
Evaluation	Error-based metrics (MSE, RMSE, R^2)	Accuracy, precision, recall, etc.

The distinguishing question is simple: **is the answer a number on a continuous scale, or a label from a fixed set of categories?** Predicting "₹75 lakh" is regression because the answer could, in principle, be any number — ₹74.8 lakh, ₹75.3 lakh, and so on, along a continuous scale of currency. Predicting "Spam / Not Spam" is classification because the answer must be exactly one of two fixed, discrete categories with no numeric ordering or in-between value. This is the single most important distinction to keep in mind for the rest of this guide.

Part 2 — NumPy and Pandas: Why They Are Used

Before touching regression, we need tools that let Python handle numbers and tables efficiently. Plain Python lists and loops are too slow and too unwieldy for the volume of arithmetic machine learning requires. NumPy and Pandas solve this.

2.1 NumPy

What NumPy is: A library that adds a fast, fixed-type, multi-dimensional array object (`ndarray`) to Python, along with a large collection of mathematical functions that operate on whole arrays at once.

Why numerical computing needs NumPy: A regular Python list stores general Python objects and loops over them one at a time in pure Python, which is slow. NumPy stores numbers in a contiguous block of memory, all of the same type, and performs operations using pre-compiled C code under the hood. This makes numeric operations dramatically faster and lets us write code that looks like math instead of loops.

NumPy arrays vs. Python lists:

	Python list	NumPy array
Element types	Mixed types allowed	Single fixed type
Element-wise math	Needs a manual loop	Built-in (`a + b`, `a * 2`)
Speed on large data	Slow	Much faster
Memory layout	Scattered objects	Contiguous block

Vectorized operations: Applying a mathematical operation to an entire array at once, without writing an explicit loop. For example, `x * 2` multiplies every element of `x` by 2 in one instruction, rather than looping element-by-element in Python.

Matrix operations: NumPy supports operations from linear algebra — matrix multiplication, transposition, inversion — which are exactly the operations used internally by regression algorithms to compute coefficients.

Shape, dimensions, rows, columns: A NumPy array has a `.shape` attribute describing its size along each axis. A 1-D array of 4 numbers has shape `(4,)`. A 2-D array with 3 rows and 2 columns has shape `(3, 2)`. Regression code frequently needs to know and control shape, because scikit-learn expects features in a specific 2-D layout (see Part 9).

Why ML libraries expect NumPy-like arrays: Scikit-learn's mathematical routines are built on top of NumPy (and libraries like it). Passing data in NumPy format lets these routines run their fast, vectorized, pre-compiled math directly, without extra conversion.

Example

```
import numpy as np

x = np.array([10, 20, 30, 40])
```

Line-by-line:

- `import numpy as np` loads the NumPy library into the program and gives it the short alias `np`, which is the near-universal convention so code stays concise.
- `x = np.array([10, 20, 30, 40])` takes a plain Python list `[10, 20, 30, 40]` and converts it into a NumPy `ndarray`, stored as a single fixed-type block of memory. `x` now supports vectorized operations, e.g. `x * 2` instantly produces `array([20, 40, 60, 80])` with no explicit loop.

2.2 Pandas

What Pandas is: A library built on top of NumPy that adds labeled, table-like data structures, designed for loading, cleaning, exploring, and manipulating real-world datasets such as CSV files.

Why DataFrames are useful: Raw NumPy arrays have no concept of column names or row labels — everything is just numbers. Real datasets have named columns ("area", "price"), mixed data types, and missing values. Pandas' `DataFrame` gives us a spreadsheet-like structure with labeled rows and columns, letting us select, filter, and clean data by name rather than by numeric position alone.

Series vs. DataFrame:

- A **Series** is a single labeled 1-D column of data (think: one spreadsheet column).
- A **DataFrame** is a 2-D table made of multiple Series sharing a common row index (think: the whole spreadsheet).

Loading a CSV file

```
import pandas as pd

df = pd.read_csv("data.csv")
```

- `import pandas as pd` loads the Pandas library under the conventional alias `pd`.
- `pd.read_csv("data.csv")` reads a comma-separated-values file from disk, parses it into rows and columns, infers a data type for each column, and returns it as a `DataFrame`, stored here in the variable `df`.

Inspecting, selecting, and filtering

```
df["price"]                    # select a single column as a Series
df[["area", "bedrooms"]]       # select multiple columns as a DataFrame
df[df["area"] > 1000]          # filter: keep only rows where area > 1000
```

- `df["price"]` selects the column named `price` and returns it as a `Series`.
- `df[["area", "bedrooms"]]` — note the double brackets — passes a *list* of column names, so the result is a `DataFrame` containing just those two columns, not a single Series.
- `df[df["area"] > 1000]` first computes a Series of `True`/`False` values (one per row, indicating whether that row's `area` exceeds 1000), then uses that Boolean Series to keep only the rows where the condition is `True`. This pattern is called **Boolean indexing** or **filtering**.

Handling missing values, dropping, and renaming

```
df.isnull().sum()                  # count missing values per column
df.dropna()                        # drop rows containing any missing value
df.drop("column_name", axis=1)     # drop a column
df.rename(columns={"old": "new"}) # rename a column
```

- `df.isnull()` produces a same-shaped DataFrame of `True`/`False` flags marking missing (`NaN`) entries; `.sum()` then counts the `True` values in each column, giving a quick per-column count of missing data.
- `df.dropna()` removes every row that has at least one missing value — useful, but potentially wasteful if it discards otherwise-good rows just because one column is missing.
- `df.drop("column_name", axis=1)` removes a column (explained fully in section 3.4).
- `df.rename(columns={"old": "new"})` returns a copy of the DataFrame with the column `"old"` renamed to `"new"`, useful for cleaning inconsistent or unclear column names from raw Kaggle data.

Basic data cleaning in Pandas essentially means iterating on these operations — inspect, filter, fix, rename, drop — until the table is consistent enough to feed into a statistical or machine-learning process.

Part 3 — Understanding a Dataset Before Building a Model

Jumping straight into modeling without first inspecting the data is one of the most common beginner mistakes. This section covers the standard first steps every practitioner runs immediately after loading a Kaggle dataset.

3.1 `head()`

```
df.head()
```

Why we use it: `head()` prints the first five rows (by default) of the DataFrame, giving an immediate, concrete look at real data rather than an abstract description.

What it shows: The actual column names as they appear in the file, sample values in each column, and the general formatting of the data (e.g., whether prices are stored as `750000` or `"₹7,50,000"` as text).

Why it's useful before modeling: It is the fastest way to catch obvious problems — misspelled or unexpected column names, values stored in the wrong format (numbers stored as text, dates stored as strings), unexpected symbols, or an entirely different structure than expected — before investing time in analysis that assumes a clean layout.

`head()` vs. `tail()`: `head()` shows the first *n* rows; `tail()` shows the last *n* rows. Checking both matters because some datasets are sorted, and problems (e.g., a block of missing values, or footer/summary rows accidentally included) may only appear at the end of the file.

3.2 `info()`

```
df.info()
```

`info()` prints a structural summary of the DataFrame:

- **Number of rows and columns**: the overall size of the dataset.
- **Column names**: a full list, useful for datasets with many columns where `head()`'s output gets truncated.
- **Data types (`dtype`)**: whether each column is stored as an integer, float, generic object (usually text), boolean, or datetime.
- **Non-null counts**: how many non-missing values exist in each column, which — compared against the total row count — immediately reveals **missing values** column by column.

Why data types matter for machine learning: Regression models require every input to be numeric. A column that *should* be numeric (like price) but was loaded as `object` (text) — often because of stray currency symbols or commas — will cause errors or silently produce nonsense if fed to a model. `info()` surfaces this immediately.

How `info()` guides preprocessing: If a column has far fewer non-null entries than the total row count, that column needs a missing-value strategy (Part 24). If

a numeric-looking column shows dtype `object`, it needs cleaning/conversion before modeling. If a column is a category (e.g. "city"), it will need encoding before use in regression.

3.3 `describe()`

```
df.describe()
```

By default, `describe()` summarizes every **numeric** column with eight statistics:

Statistic	Meaning
count	Number of non-missing values in the column
mean	Average value
std	Standard deviation (spread around the mean)
min	Smallest value
25%	First quartile — the value below which 25% of data falls
50%	Median — the middle value
75%	Third quartile — the value below which 75% of data falls
max	Largest value

Why it's used before modeling: These eight numbers give a fast statistical fingerprint of every numeric column at once, without plotting anything.

What `describe()` can reveal:

- **Missing values** — a `count` lower than the total number of rows.
- **Unusual ranges** — a minimum or maximum far outside common sense (e.g., a house `age` of -5 or 400).
- **Possible outliers** — a `max` far above the 75th percentile suggests one or a few extreme values pulling the distribution.
- **Scale differences** — one column ranging 0–1 while another ranges into the millions; this matters directly for regularized regression (Part 22).
- **Suspicious values** — placeholder codes like `-999` or `0` used to represent "unknown," which look like real data unless caught early.
- **Skewed distributions** — a large gap between the mean and median suggests the distribution is not symmetric.

Important caveat: `describe()` is an *initial* statistical inspection tool. It is not a substitute for visualizing distributions, checking relationships between variables, or a full exploratory data analysis (EDA) — it is the first pass, not the last.

3.4 `drop()`

```
df.drop("column_name", axis=1)
df.drop(index_list, axis=0)
```

Why columns/rows may need to be removed:

- A column may be irrelevant to the prediction task (e.g., an internal ID number that carries no real information about price).
- A column may leak information that would not be available at prediction time (Part 23).
- Certain rows may contain corrupted, duplicated, or otherwise unusable data.

`axis=0` vs. `axis=1`: In Pandas, `axis=0` refers to rows (moving down the index) and `axis=1` refers to columns (moving across). `df.drop("id", axis=1)` drops the column named `"id"`; `df.drop([3, 7], axis=0)` drops the rows at index positions 3 and 7.

`inplace=True`: By default, `drop()` (like most Pandas methods) returns a *new* DataFrame and leaves the original untouched. Passing `inplace=True` modifies the original DataFrame directly instead of returning a copy. This is convenient but risky, since it destroys the original data in memory — many practitioners prefer the default (non-`inplace`) behavior and reassign explicitly, e.g. `df = df.drop(...)`, so mistakes are easier to recover from by re-running earlier cells.

Why blindly dropping data is dangerous: Removing a column without understanding it might discard a genuinely predictive feature. Removing rows with `dropna()` or manual filtering can silently shrink a dataset and bias it — for example, if missingness is not random (e.g., expensive houses are more likely to have a missing "renovation year"), dropping those rows changes what the remaining data represents.

Example — dropping an irrelevant column:

```
df = df.drop("listing_id", axis=1)   # an ID number has no predictive value
```

Example — dropping unusable rows:

```
df = df.drop(df[df["price"] <= 0].index, axis=0)   # remove rows with impossible prices
```

`drop("column", axis=1)` vs. `dropna()`: `df.drop("column", axis=1)` removes an entire named column regardless of whether it has missing values. `df.dropna()` removes entire *rows* that contain at least one missing value, regardless of which column that missing value is in. They solve different problems: one is about removing a feature you've decided not to use; the other is about handling incomplete records.

Part 4 — Basic Statistics Required for Regression

Regression is built directly out of the statistical ideas below. Each one is explained with intuition, formula, a manual numeric example, Python code, and its connection to regression.

4.1 Mean

Intuition: The mean is the single number that best represents the "center" of a set of values, obtained by balancing all values against each other equally.

Formula:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

What it represents: The average value — the point where the data would "balance" if placed on a number line.

Why it's used: It's the natural reference point for measuring how far any individual value deviates from the "typical" value — and those deviations are the building block for variance, covariance, and ultimately regression.

Manual example: For $x = [2, 4, 6, 8]$: $\bar{x} = (2 + 4 + 6 + 8)/4 = 20/4 = 5$.

Python:

```
import numpy as np
x = np.array([2, 4, 6, 8])
mean_x = np.mean(x)   # 5.0
```

4.2 Median

What it represents: The middle value when data is sorted — the point that splits the data exactly in half.

Why it's useful: Unlike the mean, the median is not pulled toward extreme values.

Mean vs. median and the effect of outliers: For $[2, 4, 6, 8, 100]$, the mean is 24.0 (dragged upward by the outlier 100), but the median is 6 (unaffected). A large gap between mean and median is itself a signal of skew or outliers — this is exactly the kind of thing `describe()` (Part 3.3) helps surface.

4.3 Variance

Formula (population form):

$$Var(X) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

(The **sample variance** divides by $n - 1$ instead of n — a correction, called Bessel's correction, that produces an unbiased estimate of the true population variance when

working from a sample rather than the full population. NumPy's `np.var()` uses n by default; `ddof=1` switches to $n-1$.)

Why deviations are squared: Simply averaging the raw deviations $(x_i - \bar{x})$ always sums to zero (positive and negative deviations cancel exactly), which is useless as a measure of spread. Squaring removes the sign, so every deviation contributes positively, and larger deviations are penalized disproportionately more.

What variance measures: How spread out the values are around the mean. A variance near zero means the data is tightly clustered; a large variance means it is widely scattered.

Why variance appears in regression mathematics: Regression's least-squares coefficients (Part 7) are literally built from these same squared-deviation and cross-deviation terms — variance is the "with itself" version of the covariance calculation used to compute the slope.

Variance vs. standard deviation: Variance is in *squared* units (e.g., squared rupees), which is hard to interpret intuitively; standard deviation (below) fixes this.

Numeric example: For $x = [2, 4, 6, 8]$, $\bar{x} = 5$. Deviations: $-3, -1, 1, 3$. Squared: $9, 1, 1, 9$. Sum $= 20$. $Var(X) = 20/4 = 5$.

```
var_x = np.var(x)          # population variance, ddof=0 -> 5.0
sample_var_x = np.var(x, ddof=1)  # sample variance -> 6.666...
```

4.4 Standard Deviation

Formula:

$$\sigma = \sqrt{Var(X)}$$

Interpretation: The "typical" distance of a data point from the mean, expressed in the *original* units of the data (rupees, degrees, days) rather than squared units — this is why standard deviation is generally easier to interpret and communicate than variance.

```
std_x = np.std(x)   # sqrt(5.0) ≈ 2.236
```

4.5 Covariance

Formula:

$$Cov(X, Y) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})$$

What it tells us: Whether two variables tend to move *together* (both above their means at the same time, or both below), move *oppositely*, or show no consistent joint pattern.

- **Positive covariance**: When X is above its mean, Y also tends to be above its mean (e.g., house area and price).

- **Negative covariance**: When X is above its mean, Y tends to be below its mean (e.g., car age and resale price).
- **Near-zero covariance**: No consistent linear relationship between the two variables.

Why raw covariance is hard to compare across variables: Its magnitude depends on the units and scale of both variables — covariance between two variables measured in millions will look "big" purely because of scale, not because the relationship is strong. This limitation motivates correlation (below).

Connection to regression coefficients: As shown in Part 7, the slope of a simple linear regression is exactly the covariance of X and Y divided by the variance of X — covariance is literally half of the regression-slope formula.

Manual example: $x = [1, 2, 3, 4]$, $y = [2, 4, 5, 4]$. $\bar{x} = 2.5$, $\bar{y} = 3.75$. Deviations x: $-1.5, -0.5, 0.5, 1.5$. Deviations y: $-1.75, 0.25, 1.25, 0.25$. Products: $2.625, -0.125, 0.625, 0.375$. Sum $= 3.5$. $Cov(X, Y) = 3.5/4 = 0.875$.

```
x = np.array([1,2,3,4])
y = np.array([2,4,5,4])
cov_matrix = np.cov(x, y, ddof=0)
covariance = cov_matrix[0, 1]   # 0.875
```

4.6 Correlation (Pearson)

Formula:

$$r = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

Why normalizing by standard deviations helps: Dividing covariance by the product of the two standard deviations rescales the result so it always falls between -1 and $+1$, regardless of the original units — making it directly comparable across any pair of variables.

- r close to $+1$: strong positive linear relationship.
- r close to -1: strong negative linear relationship.
- r close to 0: weak or no *linear* relationship (a strong nonlinear relationship can still exist even when $r \approx 0$).

Strength vs. direction: The sign of r tells direction (positive/negative), and the magnitude ($|r|$) tells strength, independent of sign.

```
correlation_matrix = np.corrcoef(x, y)
r = correlation_matrix[0, 1]
```

Or with Pandas: `df[["area", "price"]].corr()`.

Correlation does NOT imply causation

Two variables can be strongly correlated without either causing the other. Classic examples:

- Ice cream sales and drowning incidents rise together — both are driven by a third factor, hot weather, not by one causing the other.
- The number of firefighters at a fire scene correlates with the amount of damage — larger fires cause both more damage *and* the dispatch of more firefighters; firefighters do not cause the damage.

This matters directly for regression: a model can find a strong linear relationship between two variables that is statistically real but causally meaningless, and using it to argue "changing X will change Y" requires much more care than fitting a regression line.

Part 5 — How Statistics Connect to Linear Regression

The concepts above are not a random detour — they build directly on top of one another:

mean $\rightarrow$ deviation from mean $\rightarrow$ variance $\rightarrow$ covariance $\rightarrow$ correlation $\rightarrow$ regression

The core intuition: If X and Y vary together in a consistent way (captured by covariance and correlation), then knowing X gives us useful information for estimating Y. Regression turns that statistical relationship into a concrete predictive formula.

Introduce the regression equation:

$$\hat{y} = b_0 + b_1 x$$

- x — the input feature (e.g., hours studied).
- y — the true, observed target value (e.g., actual exam score).
- $\hat{y}$ (read "y-hat") — the model's *predicted* value of y, to distinguish it from the true observed y.
- b_0 — the **intercept**: the predicted value of y when $x = 0$.
- b_1 — the **slope**: how much the predicted y changes for each one-unit increase in x.

Interpreting the slope: For a one-unit increase in X, the predicted Y changes by approximately b_1 units — holding other variables fixed, in the multiple-linear-regression case (Part 10) where more than one feature is involved.

Part 6 — Simple Linear Regression

Simple linear regression uses exactly one feature to predict one continuous target, fitting the best straight line through the data.

Example dataset

Hours Studied (x)	Exam Score (y)
1	35
2	42
3	50
4	58
5	65

Plotting the points: If we plot Hours Studied on the x-axis and Exam Score on the y-axis, the points trend clearly upward — more hours studied is associated with a higher score.

Finding a line: Simple linear regression searches for the single straight line $\hat{y} = b_0 + b_1 x$ that best fits this upward trend, according to a precise mathematical criterion (Part 7–8).

Prediction: Once b_0 and b_1 are known, plugging any new value of x (even one not in the original data) into the line gives a predicted score $\hat{y}$.

Residual (error):

$$e_i = y_i - \hat{y}_i$$

The residual is the vertical gap between an actual observed value y_i and the model's prediction $\hat{y}_i$ for that same input.

Why the line usually cannot pass through every point: Real data is noisy — two students who studied the same number of hours rarely get the exact same score, due to countless unmeasured factors (sleep, prior knowledge, test difficulty on the day). A single straight line has only two degrees of freedom (b_0 and b_1), so it can be tuned to fit the *overall trend* well, but generally cannot pass through every individual point exactly. The residuals are the price paid for using a simple model to summarize noisy reality.

Part 7 — Least Squares and Coefficient Calculation

7.1 The formulas

$$b_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

$$b_0 = \bar{y} - b_1\bar{x}$$

Connection to covariance and variance: The numerator of b_1 is exactly n times the covariance of X and Y; the denominator is exactly n times the variance of X. So, in words:

$$b_1 = \frac{Cov(X, Y)}{Var(X)}$$

The slope of the regression line is simply the ratio of how X and Y move together to how much X moves on its own. Once the slope is known, the intercept b_0 is chosen so that the line passes exactly through the point $(\bar{x}, \bar{y})$ — the "center of mass" of the data — which guarantees the line is correctly positioned vertically.

7.2 Complete manual calculation

Using the dataset from Part 6: $\bar{x} = 3$, $\bar{y} = 50$.

x	y	$x - \bar{x}$	$y - \bar{y}$	product	$(x - \bar{x})^2$
1	35	-2	-15	30	4
2	42	-1	-8	8	1
3	50	0	0	0	0
4	58	1	8	8	1
5	65	2	15	30	4
Sum				**76**	**10**

Slope: $b_1 = 76/10 = 7.6$

Intercept: $b_0 = 50 - (7.6)(3) = 50 - 22.8 = 27.2$

Regression equation: $\hat{y} = 27.2 + 7.6x$

Prediction for $x = 6$: $\hat{y} = 27.2 + 7.6(6) = 27.2 + 45.6 = 72.8$

Residuals and squared residuals:

x	y	$\hat{y} = 27.2 + 7.6x$	$e = y - \hat{y}$	e^2
1	35	34.8	0.2	0.04
2	42	42.4	-0.4	0.16
3	50	50.0	0.0	0.00

x	y	$\hat{y} = 27.2 + 7.6x$	$e = y - \hat{y}$	e^2
4	58	57.6	0.4	0.16
5	65	65.2	-0.2	0.04

MSE: $(0.04 + 0.16 + 0.00 + 0.16 + 0.04)/5 = 0.40/5 = 0.08$

A learner can reproduce every one of these numbers by hand using nothing but the mean and the deviation table above.

Part 8 — Why Least Squares?

Sum of Squared Errors:

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Mean Squared Error:

$$MSE = \frac{SSE}{n}$$

Why errors are squared: Exactly as with variance (Part 4.3), simply summing raw residuals cancels positive and negative errors to (near) zero, hiding how wrong the model actually is. Squaring makes every error contribute positively and penalizes large errors much more heavily than small ones, which is usually the behavior we want — a model that is wildly wrong on a few points is considered worse than one that is mildly wrong everywhere.

Why we minimize squared errors: The line defined by minimizing SSE (equivalently MSE) — the **least-squares line** — has convenient mathematical properties: it has a closed-form solution (the formulas in Part 7), it is unique, and it is the best linear unbiased estimator under classical regression assumptions (Part 25).

Geometric intuition: Picture each data point as a dot on a scatter plot and the regression line drawn through them. Each residual e_i is the *vertical* distance from a point straight up or down to the line (not the perpendicular distance). Least squares finds the one line that makes the sum of the squares of all these vertical distances as small as possible.

Part 9 — Python Implementation of Linear Regression

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([35, 42, 50, 58, 65])

model = LinearRegression()

model.fit(X, y)

prediction = model.predict([[6]])

print(prediction)
print(model.coef_)
print(model.intercept_)
```

Line-by-line explanation:

- `import numpy as np` — NumPy is needed because scikit-learn expects numeric input in array form, and we build our feature and target arrays with it.
- `from sklearn.linear_model import LinearRegression` — imports the specific class that implements ordinary least-squares linear regression from scikit-learn's collection of linear models.
- `X = np.array([[1], [2], [3], [4], [5]])` — **why the double brackets**: scikit-learn always expects features as a 2-D array of shape `(n_samples, n_features)` — rows are samples, columns are features — even when there is only one feature. Writing `[[1], [2], [3], [4], [5]]` creates a 2-D array with shape `(5, 1)`: 5 samples, 1 feature. Writing plain `[1,2,3,4,5]` would create a 1-D array of shape `(5,)`, which scikit-learn's `.fit()` will reject for `X`.
- `y = np.array([35, 42, 50, 58, 65])` — **why y has a different shape**: the target is expected as a 1-D array of shape `(n_samples,)` — one number per sample — not a 2-D column, so single brackets are correct here.
- `model = LinearRegression()` — creates an *untrained* instance of the linear regression algorithm. At this point `model` has no learned coefficients yet; it is only configured with (default) settings.
- `model.fit(X, y)` — this is where training actually happens. Internally, scikit-learn solves the least-squares equations from Part 7 (generalized to matrix form for any number of features) to find the values of b_0 and b_1 that **minimize the sum of squared residuals** between the model's predictions and the true y values on this training data. Concretely: "training" means "solving an optimization problem to find the parameter values that minimize a loss function" — here, the loss is SE, and because ordinary least-squares linear regression has a closed-form solution, this can be computed directly via matrix algebra rather than iterative search.
- `model.predict([[6]])` — applies the now-learned b_0 and b_1 to a brand-new input (6 hours studied, never seen during training) and returns $\hat{y} = b_0 + b_1(6)$.

- `model.coef_` — the learned slope(s) b_1 (an array, one entry per feature). For this dataset it should be very close to 7.6, matching the manual calculation in Part 7.
- `model.intercept_` — the learned intercept b_0, here very close to 27.2.

This example is not a coincidence: scikit-learn's `LinearRegression.fit()` is solving exactly the same least-squares problem we solved by hand in Part 7, just generalized to work with any number of features and any number of samples.

Part 10 — Multiple Linear Regression

Why one feature is sometimes insufficient: A house's price depends on far more than just its area — bedrooms, age, location, and other factors all contribute. Using only one feature throws away useful information and typically produces a much less accurate model.

Formula:

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_p x_p$$

- Each x_j is a separate feature (area, bedrooms, age, ...).
- Each b_j is that feature's own coefficient, learned independently.
- b_0 is still the intercept — the predicted value when every feature equals zero.

Interpreting a coefficient — "holding other variables constant": b_1 represents the expected change in $\hat{y}$ for a one-unit increase in x_1, *assuming all other features stay fixed*. This caveat matters: in reality, features are rarely independent of each other (see multicollinearity, Part 26), so this interpretation is a mathematical idealization, not always literally achievable in the real world.

Python

```
X = df[["area", "bedrooms", "age"]]
y = df["price"]

model = LinearRegression()
model.fit(X, y)

prediction = model.predict([[1800, 3, 5]])
```

- `X = df[["area", "bedrooms", "age"]]` selects three columns as a DataFrame — note this naturally forms a 2-D table, exactly the shape scikit-learn expects, with no manual reshaping needed (unlike the single-feature NumpPy example in Part 9).
- `y = df["price"]` selects the target as a 1-D Series.
- `model.fit(X, y)` solves for four numbers at once: one intercept and one coefficient per feature, again by minimizing SSE across all training rows.
- `model.predict([[1800, 3, 5]])` predicts the price for a new house with area 1800, 3 bedrooms, and age 5 years — the order of the values must match the column order used in training (`area`, `bedrooms`, `age`).

Simple vs. multiple linear regression

	Simple Linear Regression	Multiple Linear Regression
Number of features	Exactly 1	2 or more
Formula	$\hat{y} = b_0 + b_1 x$	$\hat{y} = b_0 + b_1 x_1 + \cdots + b_p x_p$

	Simple Linear Regression	Multiple Linear Regression
Geometric shape	A line in 2-D	A flat hyperplane in higher dimensions
Coefficient interpretation	Slope of the line	Effect of one feature, others held fixed
Typical use case	One clearly dominant predictor	Most real-world problems

Multiple linear regression is not a different mathematical family from simple linear regression — it is the *same* least-squares linear model, just extended to use more than one predictor at once (Part 12 revisits this point directly).

Part 11 — Polynomial Regression

Why a straight line cannot represent every relationship: Many real relationships bend or curve rather than following a straight line — experience and salary often show diminishing returns at higher experience levels; electricity demand rises sharply as temperature moves to either extreme; fuel consumption rises non-linearly with speed. Forcing a straight line onto a curved relationship produces systematically wrong predictions at the extremes.

Formula:

$$\hat{y} = b_0 + b_1 x + b_2 x^2 + b_3 x^3 + \cdots + b_d x^d$$

- **Degree (d)**: the highest power of x used. Degree 1 is just ordinary linear regression; degree 2 adds a squared term allowing one bend (a parabola); degree 3 allows an S-shaped curve, and so on.
- **Polynomial features**: the extra columns $x^2, x^3, \ldots$ engineered from the original x before fitting.

Why the shape becomes nonlinear in x, yet the model is still "linear": This is a subtle but important distinction. "Linear" in *linear regression* refers to linearity **in the coefficients** b_j — the prediction is a straight-line (weighted-sum) combination of the input columns, whatever those columns happen to contain. If we manufacture new columns x^2, x^3, etc., and then fit $\hat{y} = b_0 + b_1 x + b_2 x^2 + b_3 x^3$, this is still a *linear combination* of the columns $\{1, x, x^2, x^3\}$ — the coefficients still enter additively and are still solved for using the same least-squares machinery as before. The resulting *curve* in x-vs-y space is nonlinear, but the *model with respect to its parameters* is linear. This is why polynomial regression is implemented simply as "linear regression on transformed features," not as a separate algorithm.

Python

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("poly", PolynomialFeatures(degree=2)),
    ("linear", LinearRegression())
])

model.fit(X, y)
prediction = model.predict([[6]])
```

- `PolynomialFeatures(degree=2)` is a transformer that takes the original feature(s) and generates new columns for every power up to the given degree (and, with multiple original features, their interaction terms too).
- `Pipeline([...])` chains multiple processing steps into a single object that behaves like one model: calling `.fit()` on the pipeline first applies `PolynomialFeatures` to expand X, *then* fits `LinearRegression` on the expanded features. Calling

`.predict()` automatically applies the same transformation to new data before predicting.

- **Why a pipeline is useful**: Without it, we would have to manually transform training data, remember to apply the *exact same* transformation to test data and to any future new data, and keep the transformer and the model in sync by hand — a common source of bugs. A pipeline guarantees the same steps happen, in the same order, every time.

Degree 1, 2, 3, and high-degree polynomials:

- **Degree 1** is a straight line — no curvature.
- **Degree 2** allows one bend — good for relationships with a single curve or turning point.
- **Degree 3** allows an S-shaped curve — two turning points.
- **Very high degrees** can bend and wiggle enough to pass extremely close to *every* training point.

The danger of very high polynomial degrees: A high-degree polynomial has enough flexibility to snake through the noise in the training data rather than capturing the true underlying trend. It will look excellent on training data (very low training error) but will oscillate wildly and generalize very poorly to new data. This is the clearest, most visual introduction to **overfitting**, explored fully in Part 18.

Part 12 — Comparing the Three Regression Types

Model	Formula	Shape	Features	When to Use	Main Risk
Linear	$\hat{y} = b_0 + b_1 x$	Straight line	One	A single feature has a clear, roughly straight-line relationship with the target	Underfitting if the true relationship curves
Multiple Linear	$\hat{y} = b_0 + \sum b_j x_j$	Flat hyperplane	Many	Several features jointly and roughly linearly explain the target	Underfitting (if relationships curve) or multicollinearity among features
Polynomial	$\hat{y} = b_0 + \sum b_j x^j$	Curve	Usually one feature, transformed	The relationship visibly bends or curves	Overfitting at high degrees

Multiple linear regression is linear regression with more predictors — not a separate family. Both simple and multiple linear regression fit a model that is linear in its coefficients and solved by the same least-squares mathematics; the only difference is how many feature columns are involved. Polynomial regression is a special case of this same idea: it is still "linear regression," just applied to columns that happen to be powers of an original feature.

Why choose one model over another: Start simple. Try linear (or multiple linear) regression first — it is easy to interpret and hard to overfit. Add polynomial terms only if a scatter plot or residual pattern clearly shows curvature that a straight line/hyperplane cannot capture, and even then, prefer the lowest degree that fixes the visible curvature (Parts 18–19 explain exactly how to judge this using held-out data, not just the shape of the training-data fit).

Part 13 — Prediction

Once any of the three model types has been trained, using it to predict follows the same conceptual pipeline:

$X_{new} \rightarrow$ model applies learned coefficients $\rightarrow$ calculate $\hat{y} \rightarrow$ return predicted numerical valu

1. Linear regression example: Using $\hat{y} = 27.2 + 7.6x$ from Part 7, for a new $x = 7$ (hours studied): $\hat{y} = 27.2 + 7.6(7) = 27.2 + 53.2 = 80.4$.

2. Multiple linear regression example: Suppose a fitted model gives $\hat{y} = 50000 + 300\,x_1(\text{area}) - 2000\,x_2(\text{age})$. For a new house with area 1200 and age 10: $\hat{y} = 50000 + 300(1200) - 2000(10) = 50000 + 360000 - 20000 = 390000$.

3. Polynomial regression example: Suppose a fitted degree-2 model gives $\hat{y} = 10 + 2x + 0.5x^2$. For $x = 4$: $\hat{y} = 10 + 2(4) + 0.5(16) = 10 + 8 + 8 = 26$.

Training vs. inference (prediction): **Training** is the one-time (or periodic) process of using known input–output pairs to solve for the model's coefficients ($b_0, b_1, ...$) by minimizing error. **Inference**, also called prediction, is the repeated, ongoing process of plugging *new* inputs into an *already-trained* model's formula to get an output — no error-minimization or learning happens at this stage, only arithmetic using coefficients that are already fixed.

Part 14 — Train/Test Split

Why we cannot evaluate a model only on the data it trained on: A model can achieve near-perfect performance on its training data simply by fitting closely to that data's specific quirks and noise, without having learned anything generalizable. Testing on the same data used for training would report an overly optimistic score that says nothing about how the model will perform on new, real-world inputs it hasn't seen. This is the practical reason underfitting and overfitting (Part 18) must be diagnosed using data the model has never touched during training.

- **Training set**: The portion of data used to fit the model's parameters.
- **Testing set**: A portion held back and used only afterward, to measure how well the model generalizes.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Every parameter explained:

- `X`, `y` — the full feature table and target array to be split.
- `test_size=0.2` — reserves 20% of the rows for testing and uses the remaining 80% for training. This ratio is a common convention (other common splits include 70/30 or 75/25) chosen to leave enough data for training while still keeping a meaningfully sized, statistically useful test set.
- `random_state=42` — fixes the internal random-number generator's starting point so the *same* random split is produced every time the code runs. This is purely for **reproducibility** — so a colleague running the identical code gets identical train/test rows and can exactly reproduce your results — the number 42 has no special meaning; any fixed integer works.
- **Why random splitting is useful**: It avoids any hidden ordering in the raw data (e.g., rows sorted by price, or by date) from systematically ending up all in one split, which could make training or testing unrepresentative of the whole dataset.
- **When random splitting may not be appropriate**: For **time-series data** (e.g., predicting tomorrow's demand from historical data), randomly shuffling rows before splitting can leak future information into the training set and let the model "peek" at patterns from dates it should not yet know about. Time-series problems generally require a **chronological split** instead — training on earlier time periods and testing on strictly later ones.

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Why predictions should be made on `X_test`: The entire purpose of the held-out

test set is to measure performance on data the model never used while learning its coefficients. Predicting on `X_train` (or on the full dataset) would reintroduce the exact problem this section exists to avoid.

Part 15 — Model Evaluation

Why generating predictions is not enough: A raw list of predicted numbers next to actual numbers doesn't, by itself, say whether the model is doing a good job — we need a single, well-defined number that summarizes overall accuracy so different models (or different settings of the same model) can be compared objectively.

15.1 Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Calculation**: average of the squared residuals across all test samples.
- **Interpretation**: the average squared prediction error — smaller is better.
- **Units**: squared units of the target (e.g., squared rupees), which is not directly intuitive.
- **Sensitivity to large errors**: because errors are squared, a few very large mistakes inflate MSE disproportionately more than many small ones — MSE is highly sensitive to outlier errors.
- **Advantages**: mathematically convenient (differentiable, connects directly to least squares), heavily penalizes large mistakes when that is desirable.
- **Disadvantages**: hard to interpret directly because of squared units; can be dominated by a small number of bad predictions.

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_test, y_pred)
```

- `mean_squared_error(y_test, y_pred)` computes the formula above directly, comparing each true value in `y_test` to the model's corresponding prediction in `y_pred`.

15.2 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

- **Why the square root is taken**: it converts the error back from squared units into the *original* units of the target (rupees, degrees, minutes), undoing the squaring step from MSE's formula.
- **Interpretation**: RMSE can be read as "roughly how far off, on average, the model's predictions are, in the same units as the target" (technically, it is closer to a root-mean-square average of the errors rather than a simple average, but the intuitive reading is close enough for practical use).
- **Why it's easier to communicate**: saying "the model is off by about ₹2,00,000 on average" (RMSE) is far more meaningful to a non-technical audience than "the model has an MSE of 40,000,000,000."

```
import numpy as np
rmse = np.sqrt(mse)
```

15.3 R-squared (Coefficient of Determination)

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$SS_{res} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad SS_{tot} = \sum_{i=1}^{n}(y_i - \bar{y})^2$$

- SS_{res} is the total squared error of the model's predictions (identical in spirit to the numerator of MSE).
- SS_{tot} is the total squared error of the simplest possible "model" — one that always predicts the mean $\bar{y}$ for every input, regardless of features. This is the **baseline** R^2 is measured against.

Interpretation: R^2 measures what fraction of the variance in the target the model explains, relative to just guessing the mean every time.

- $R^2 = 1$: the model's predictions exactly match every true value (zero error).
- $R^2 = 0$: the model performs exactly as well as always predicting the mean — it has learned nothing useful from the features.
- **Negative** R^2: possible on test data (though not on the training data used to fit an ordinary least-squares model) — it means the model's predictions are *worse* than simply guessing the mean every time, a red flag that something is badly wrong (overfitting, a poor feature set, or a bug).

Why a high R^2 doesn't automatically mean the model is good: R^2 can be inflated by overfitting (especially with many features or high polynomial degree) or can be high while the model still has practically large, unacceptable errors on a target with wide natural scale. It should always be read together with RMSE and a sanity check on typical error size in real-world terms.

Why R^2 should not be read as "percent accuracy": $R^2 = 0.85$ does *not* mean "the model is 85% accurate" in the everyday sense of getting 85% of predictions exactly right — it means the model explains 85% of the target's variance relative to the mean-only baseline. This is a fundamentally different (and more technical) statement than "accuracy."

```
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)
```

- `r2_score(y_test, y_pred)` computes the formula above, comparing the model's errors against the mean-only baseline's errors on the same test data.

Part 16 — MSE vs. RMSE vs. R²

	MSE	RMSE	R²
Formula	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	$\sqrt{MSE}$	$1 - \frac{SS_{res}}{SS_{tot}}$
Meaning	Average squared error	Typical error size, original units	Fraction of variance explained vs. mean baseline
Units	Squared target units	Same units as target	Unitless (0–1 scale, can go negative)
Sensitive to outliers?	Very	Very (inherits from MSE)	Indirectly, via SS_{res}
Better when?	Comparing models mathematically, e.g. inside optimization	Communicating typical error to humans	Comparing how much of the variance is explained
Lower or higher better?	Lower is better	Lower is better	Higher is generally better, but context matters
Common mistakes	Reading it directly as "typical error" (units are squared)	Treating it as the *maximum* error rather than a typical one	Reading it as "percent accuracy"; comparing across different target definitions or different test sets

MSE and RMSE: lower is always better — zero would mean perfect predictions.

R²: generally, higher is better, but context matters — a very high R^2 on a small or unusually clean test set can be misleading, and different domains have very different "good" thresholds (an R^2 of 0.6 might be excellent for predicting human behavior but poor for predicting a controlled physical process).

Why comparisons must use the same test set and target definition: MSE, RMSE, and R² are only meaningful *relative* to a specific target and a specific test set. Comparing Model A's RMSE on one test set to Model B's RMSE on a *different* test set (different rows, or a differently defined or differently scaled target) produces a number that looks like a fair comparison but is not — always evaluate competing models on the identical held-out test set and identical target definition.

Part 17 — Evaluating All Three Regression Models

Using one small dataset (e.g., house price predicted from area, with an engineered polynomial term on area), the workflow is the same for each model type:

```
# Linear Regression (single feature)
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
pred_linear = linear_model.predict(X_test)

# Multiple Linear Regression (several features)
multi_model = LinearRegression()
multi_model.fit(X_train_multi, y_train)
pred_multi = multi_model.predict(X_test_multi)

# Polynomial Regression (degree 2)
poly_model = Pipeline([("poly", PolynomialFeatures(degree=2)),
                       ("linear", LinearRegression())])
poly_model.fit(X_train, y_train)
pred_poly = poly_model.predict(X_test)
```

For each model, compute all three metrics on the *same* `X_test`/`y_test`:

```
for name, preds in [("Linear", pred_linear), ("Multiple", pred_multi), ("Polynomial", p
    mse = mean_squared_error(y_test, preds)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, preds)
    print(name, mse, rmse, r2)
```

Example comparison table (illustrative numbers)

Model	MSE	RMSE	R^2
Linear	45,000,000	6,708	0.71
Multiple Linear	28,000,000	5,292	0.84
Polynomial (deg 2)	25,500,000	5,050	0.86

Deciding which model is better: Prefer the model with the lowest RMSE/MSE and highest R^2 *on the test set*, but weigh this against interpretability and robustness — a marginally better polynomial fit is not worth adopting if it is only slightly better on this particular test split but noticeably less stable or interpretable overall (a concern examined directly in the next two parts).

Why one metric alone is not enough: A model could show a low MSE/RMSE purely because the target has a small natural scale, while its R^2 reveals it barely beats the mean-only baseline. Conversely, a high R^2 can coexist with an RMSE that is still practically unacceptable for the application. Looking at all three together — and comparing training performance to test performance — gives a much fuller picture than any single number.

Part 18 — Underfitting and Overfitting

Underfitting

- The model is **too simple** to capture the real pattern in the data (e.g., fitting a straight line to a clearly curved relationship).
- This is described as **high bias** — the model's assumptions are too rigid to represent reality.
- Result: **poor performance on training data** *and* poor performance on test data — the model fails everywhere, because it never learned the actual pattern in the first place.

Overfitting

- The model is **too flexible** and starts fitting the noise and idiosyncrasies specific to the training data, rather than the true underlying trend.
- Result: **very good (even near-perfect) performance on training data**, but **poor performance on test data** — the model has effectively memorized training quirks that do not generalize.
- This is described as **high variance** — small changes in the training data would produce a very different fitted model, because the model's shape depends heavily on the specific noise it saw.

Demonstrating overfitting with polynomial regression

Degree	Training fit	Test fit	Diagnosis
1	Poor (misses curvature)	Poor	Underfitting
2	Good	Good	Good balance
5	Very good	Starts to degrade	Beginning to overfit
15	Near-perfect (wiggles through nearly every point)	Very poor (wild swings between training points)	Severe overfitting

Why increasing complexity is not always better: A more complex, more flexible model can always fit the training data at least as well as a simpler one — but that flexibility is a double-edged sword. Beyond a certain point, additional complexity is spent modeling *noise specific to this particular training sample*, not the general pattern, which actively hurts performance on any new data.

Part 19 — Bias–Variance Tradeoff

- **Bias**: error introduced by approximating a real, possibly complex relationship with a model that is too simple. High bias → underfitting.
- **Variance**: error introduced by the model being overly sensitive to the specific training data it happened to see. High variance → overfitting.

Model complexity	Bias	Variance	Typical symptom
Too simple	High	Low	Underfitting: poor on both train and test
Well balanced	Moderate	Moderate	Good generalization
Too complex	Low	High	Overfitting: great on train, poor on test

The tradeoff: As model complexity increases, bias tends to fall (the model can represent more complex patterns) while variance tends to rise (the model becomes more sensitive to training-data noise). Total expected error is roughly the sum of (squared) bias, variance, and irreducible noise — so the *best* model is not the simplest or the most complex, but the one that minimizes the combination of bias and variance for a given dataset.

Why machine learning is fundamentally about this balance: Nearly every modeling decision in this guide — choosing polynomial degree, choosing regularization strength (Part 20), deciding whether to add more features — is, underneath, a decision about where to sit on this bias–variance spectrum. There is no universally "correct" complexity; it depends on how much data is available, how noisy it is, and how complex the true underlying relationship actually is. Held-out test performance (not training performance) is the tool used to find a good balance in practice.

Part 20 — Regularization

Why regularization is needed: Left unconstrained, especially with many features or high-degree polynomial terms, a linear model can assign very large coefficients to fit training data closely — a hallmark of overfitting (Part 18). Regularization directly discourages this by adding a penalty for coefficient size into the training objective, trading a small amount of training fit for a model that generalizes better.

Basic idea: Penalize overly large coefficients so the model does not become unnecessarily complex, even if a more complex fit would slightly reduce training error.

20.1 Ridge Regression (L2 penalty)

$$\text{Loss} = SSE + \lambda \sum_{j=1}^{p} \beta_j^2$$

- λ (in scikit-learn, the parameter is called `alpha`) controls the **strength** of the penalty: $\lambda = 0$ reduces Ridge to ordinary least squares; larger λ pushes coefficients more aggressively toward zero.
- **Coefficient shrinkage**: because large coefficients are penalized (squared, so large ones are penalized disproportionately), the optimizer prefers a solution with smaller, more conservative coefficients, even if it fits the training data slightly less tightly.
- **Effect of increasing regularization**: as λ grows, coefficients shrink continuously toward — but generally not exactly to — zero, and the model becomes progressively simpler and less sensitive to any single feature or to noise.
- **Why Ridge generally doesn't force coefficients to exactly zero**: the squared penalty (β_j^2) shrinks large coefficients hard but shrinks near-zero coefficients only very gently (the derivative of β^2 vanishes as $\beta \to 0$), so it rarely drives them all the way to exactly zero — it shrinks everything smoothly but keeps every feature in the model to some degree.

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
```

- `Ridge(alpha=1.0)` creates a Ridge regression model with penalty strength $\lambda = 1.0$ — a starting point that is often tuned via experimentation or cross-validation.
- `model.fit(X_train, y_train)` solves the penalized least-squares problem above (still with a closed-form matrix solution for Ridge) instead of plain OLS.

20.2 Lasso Regression (L1 penalty)

$$\text{Loss} = SSE + \lambda \sum_{j=1}^{p} |\beta_j|$$

- **L1 penalty**: uses the *absolute value* of each coefficient rather than its square.

- **Coefficient shrinkage and exact zeros**: unlike Ridge's squared penalty, the absolute-value penalty has a constant "pull" toward zero regardless of how small a coefficient already is, which is mathematically capable of pushing some coefficients all the way to **exactly zero** — effectively removing those features from the model entirely.
- **Feature selection**: because Lasso can zero out coefficients completely, it doubles as an automatic feature-selection method — the surviving nonzero coefficients indicate which features the model considers useful.
- **Effect of alpha**: as with Ridge, increasing `alpha` increases the strength of the penalty; with Lasso, larger `alpha` also tends to zero out *more* features, producing a progressively sparser model.

```
from sklearn.linear_model import Lasso

model = Lasso(alpha=1.0)
model.fit(X_train, y_train)
```

- Identical structure to the Ridge example, but `Lasso` solves the L1-penalized objective, generally via an iterative optimization algorithm rather than a single closed-form matrix formula.

Part 21 — Ridge vs. Lasso

Property	Ridge	Lasso
Penalty	L2 ($\sum \beta_j^2$)	L1 ($\sum \lvert\beta_j\rvert$)
Formula	$SSE + \lambda \sum \beta_j^2$	$SSE + \lambda \sum \lvert\beta_j\rvert$
Shrinks coefficients	Yes	Yes
Can set coefficients exactly to zero	Generally no	Yes
Useful for feature selection	Limited	Yes
Handling correlated predictors	Tends to shrink correlated coefficients together, keeping all of them	Tends to arbitrarily pick one of a correlated group and zero out the others
Main use	Reducing overfitting while keeping all features, especially with many small/moderate correlated predictors	Reducing overfitting *and* simplifying the model by dropping less useful features

When to choose Ridge vs. Lasso: Choose Ridge when most features are believed to contribute at least a little, and the priority is stabilizing coefficients (especially under multicollinearity, Part 26) without discarding any feature outright. Choose Lasso when it is suspected that only a subset of features is genuinely useful and an interpretable, sparser model is preferred.

Elastic Net: A combination of both penalties, $\lambda_1 \sum \lvert\beta_j\rvert + \lambda_2 \sum \beta_j^2$, giving some of Lasso's feature-selection behavior along with some of Ridge's stability when predictors are correlated — a practical middle ground when it's unclear in advance which of the two penalties suits the data better.

Part 22 — Feature Scaling

Why feature scale matters: Consider:

- `age` = 25
- `salary` = 800000
- `area` = 1500

These features live on wildly different numeric scales. A regularization penalty like Ridge's or Lasso's is applied to the raw coefficient values — but a feature measured in the hundreds of thousands (salary) will naturally need a *tiny* coefficient to have a sensible effect on the prediction, while a feature measured in single digits (e.g., number of bedrooms) will need a much *larger* coefficient. Since the penalty term treats all coefficients equally regardless of what they multiply, unscaled features cause the regularization to punish some features far more harshly than others simply because of their units — not because they are actually less important.

- **Standardization**: transforms each feature to have a mean of 0 and a standard deviation of 1, putting all features on a comparable scale before any coefficient-size penalty is applied.
- **Why scaling is especially important for Ridge/Lasso**: without it, regularization strength effectively becomes arbitrary and inconsistent across features, distorting which features get shrunk or zeroed out.
- **Why ordinary least-squares linear regression is less sensitive to scaling**: OLS's *predictions* are mathematically unaffected by rescaling a feature (the coefficient simply rescales to compensate), so plain linear regression does not strictly require scaling to produce correct predictions. Scaling can still help numerical stability of the solving algorithm and makes coefficients directly comparable to each other, but it is not a correctness requirement for OLS the way it is for regularized models.

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge

model = Pipeline([
    ("scaler", StandardScaler()),
    ("ridge", Ridge(alpha=1.0))
])
model.fit(X_train, y_train)
```

`StandardScaler` learns the mean and standard deviation of each feature *from the training data only* and applies the transformation `(x - mean) / std`; wrapping it in a `Pipeline` with the model guarantees the exact same scaling (learned only from training data) is automatically applied to any test or new data passed through `.predict()` — this connects directly to the next section.

Part 23 — Data Leakage

Definition: Data leakage occurs when information that would not actually be available at prediction time improperly enters the training process, making the model's evaluated performance look better than it will truly be in production.

Examples:

- Scaling (`StandardScaler`) or imputing missing values using statistics computed from the *entire* dataset (train + test combined) before splitting — the test set's information subtly influences the "training" process, inflating apparent performance.
- Including a feature that is only known *after* the outcome occurs (e.g., using "days until loan default" as a feature to predict *whether* a loan defaults).
- Accidentally including the target variable itself, or a near-duplicate of it, disguised as a feature.

Why preprocessing should be fit only on training data: Any transformation that "learns" something from the data (a scaler's mean/std, an imputer's fill value, an encoder's categories) must be fit exclusively using the training set, then applied unchanged to the test set. Fitting it on the combined data lets test-set values indirectly influence how training data gets transformed, contaminating the independence of the test set.

Correct pipeline approach:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("ridge", Ridge(alpha=1.0))
])

pipeline.fit(X_train, y_train)      # scaler's mean/std learned ONLY from X_train
predictions = pipeline.predict(X_test)  # X_test is scaled using X_train's learned stat
```

Why pipelines are useful (revisited): A `Pipeline` automatically enforces the correct order: `.fit()` fits the scaler using only the data passed to it (the training set), and `.predict()`/`.transform()` on new data reuses those already-learned statistics rather than recomputing them — making leakage-free preprocessing the *default* behavior rather than something that has to be remembered and done manually every time.

Part 24 — Missing Values and Categorical Features

Real Kaggle datasets routinely contain `NaN` (missing) values, text strings, category labels, and columns that are simply not useful — all of which must be handled before a regression model (which only accepts numbers, with no missing entries) can be trained.

- **`SimpleImputer`**: fills in missing values using a chosen strategy (e.g., the column's mean, median, or a constant), so no `NaN` values remain.
- **`OneHotEncoder`**: converts a categorical column (e.g., `"city"` with values like `"Mumbai"`, `"Delhi"`, `"Pune"`) into multiple binary (0/1) columns — one per category — since regression cannot directly use text labels.
- **`ColumnTransformer`**: applies different preprocessing steps to different columns in a single object — e.g., imputing and scaling numeric columns while separately imputing and one-hot-encoding categorical columns — combining everything into one coherent transformation step.
- **`Pipeline`**: as before, chains the `ColumnTransformer` together with the final regression model into one object that can be fit and used for prediction as a single unit.

Example realistic preprocessing + regression pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LinearRegression

numeric_features = ["area", "age"]
categorical_features = ["city"]

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

full_pipeline = Pipeline([
    ("preprocessing", preprocessor),
```

```
    ("model", LinearRegression())
])

full_pipeline.fit(X_train, y_train)
predictions = full_pipeline.predict(X_test)
```

This single object handles missing numeric values, scales numeric columns, handles missing categorical values, one-hot-encodes categories, and fits a regression model — all in the correct, leakage-free order, and all reusable on new data with one `.predict()` call.

Part 25 — Common Regression Assumptions

Ordinary linear regression's classical statistical theory rests on several assumptions:

- **Linearity**: the true relationship between features and target is (approximately) a straight-line/hyperplane combination.
- **Independence**: residuals for different observations are not correlated with each other (violated, for example, by time-series data with autocorrelation).
- **Constant variance / homoscedasticity**: the spread of residuals should be roughly the same across all levels of the predicted value, not systematically wider for some ranges than others (the opposite — increasing or uneven spread — is called heteroscedasticity).
- **Residual behavior**: residuals should show no obvious remaining pattern when plotted against predictions or features — a visible pattern suggests the model is missing some structure in the data (e.g., unaccounted-for curvature).
- **Multicollinearity**: predictors should not be too strongly correlated with each other (Part 26).
- **Normality of residuals**: residuals should be approximately normally distributed — this assumption matters specifically for classical statistical inference (constructing confidence intervals and hypothesis tests on the coefficients), not for the point predictions themselves.

Assumptions for reliable statistical inference vs. requirements for prediction: It's important to distinguish two different goals. If the goal is to make trustworthy statements like "we are 95% confident the true effect of area on price lies between X and Y," the classical assumptions above matter a great deal, because the formulas for confidence intervals and p-values are derived assuming they hold. If the goal is purely predictive — get the best possible $\hat{y}$ for new inputs, as measured by held-out RMSE/R^2 — violating some of these assumptions (especially normality of residuals) does not necessarily prevent a linear regression model from being useful; scikit-learn's `LinearRegression`, for instance, does not require normal residuals to produce reasonable point predictions. It would be incorrect to claim that *every* assumption listed above is an absolute prerequisite for *every* predictive use of regression — the assumptions matter most, and most strictly, for inference about the coefficients themselves.

Part 26 — Multicollinearity

What happens when predictors are strongly correlated with each other? Consider predicting house price using:

- house area
- number of rooms
- number of bedrooms

These three are naturally strongly correlated (a bigger house tends to have more rooms and bedrooms). This is called **multicollinearity**.

Why coefficient interpretation becomes unstable: When two or more features move together almost in lockstep, the regression math struggles to determine how much of the target's variation to attribute to each one individually — small changes in the data (or even in floating-point computation) can swing the coefficients substantially, sometimes even flipping their sign, even though the model's overall *predictions* may remain fairly stable. This mainly damages the *interpretability* of individual coefficients, more than the raw predictive accuracy of the model as a whole.

Why correlated predictors create problems: The least-squares formulas (Part 7, generalized to multiple features via matrix algebra) rely on inverting a matrix built from the features; when features are highly correlated, this matrix becomes close to non-invertible ("ill-conditioned"), which is the mathematical root of the coefficient instability described above.

Variance Inflation Factor (VIF): An optional diagnostic statistic computed for each feature that measures how much that feature's variance is "inflated" due to its correlation with the other features. A commonly cited (though not universal) rule of thumb treats VIF values above roughly 5–10 as a signal of concerning multicollinearity, worth investigating further.

How Ridge can help: Ridge regression's L2 penalty (Part 20) directly stabilizes the coefficient-estimation math in the presence of correlated features, because it adds a term to the matrix that is being inverted, making that matrix better-conditioned and the resulting coefficients much less sensitive to small data changes — one of Ridge's most common practical uses is precisely to handle multicollinearity gracefully.

Part 27 — Visualization

Numerical metrics (MSE, RMSE, R^2) tell us *how much* error exists, but not *what kind* of error, or *where* it occurs — visualization fills that gap and often reveals problems a single number cannot (e.g., a specific range of inputs where the model consistently underpredicts).

Simple linear regression

```
import matplotlib.pyplot as plt

plt.scatter(X_test, y_test, color="blue", label="Actual")
plt.plot(X_test, y_pred, color="red", label="Predicted line")
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.legend()
plt.show()
```

- `plt.scatter(...)` plots the true data points, so we can see the real spread of the data.
- `plt.plot(X_test, y_pred, ...)` draws the model's fitted line/predictions directly over the same axes, making it visually obvious how well the line tracks the actual points.

Residual plot

```
residuals = y_test - y_pred
plt.scatter(y_pred, residuals)
plt.axhline(y=0, color="black", linestyle="--")
plt.xlabel("Predicted values")
plt.ylabel("Residuals")
plt.show()
```

A good model's residual plot shows points scattered randomly around the zero line with no visible pattern. A clear pattern (e.g., a curve, or a funnel shape that widens) signals, respectively, unmodeled nonlinearity or heteroscedasticity (Part 25).

Polynomial regression

```
import numpy as np

x_range = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
y_curve = poly_model.predict(x_range)

plt.scatter(X, y, color="blue", label="Actual")
plt.plot(x_range, y_curve, color="red", label="Polynomial fit")
```

```
plt.legend()
plt.show()
```

- `np.linspace(X.min(), X.max(), 200)` generates 200 evenly spaced values across the range of X, purely so the fitted curve can be drawn smoothly rather than only at the handful of original data points.
- `.reshape(-1, 1)` converts this 1-D array into the 2-D shape scikit-learn expects for features (recall Part 9) — the `-1` tells NumPy to automatically infer the correct number of rows given 1 column.

Why visualization should accompany numerical metrics, not replace them: A chart makes patterns (curvature, funnel-shaped residuals, a poorly fit region) immediately visible to a human, while metrics give precise, comparable numbers that a chart alone cannot provide (e.g., precisely comparing two models with visually similar-looking fits). Good practice uses both together.

Part 28 — Complete Kaggle Workflow

This section assembles every previous part into one reusable, end-to-end template — the sequence a practitioner follows from a raw downloaded CSV to a compared, evaluated set of regression models.

```
# 1. Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load the data
df = pd.read_csv("data.csv")

# 3. First look at the data (Part 3)
df.head()
df.info()
df.describe()

# 4. Basic cleaning (Part 3)
df = df.drop("id", axis=1)                      # remove irrelevant column
df = df.drop(df[df["price"] <= 0].index)        # remove impossible rows

# 5. Exploratory statistics (Part 4)
correlations = df.corr(numeric_only=True)

# 6. Define features and target
numeric_features = ["area", "age"]
categorical_features = ["city"]
X = df[numeric_features + categorical_features]
y = df["price"]

# 7. Train/test split (Part 14) — done BEFORE fitting any preprocessing, to avoid leaka
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 8. Preprocessing pipeline (Part 24)
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
```

```
    ("scaler", StandardScaler())
])
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

# 9. Build and fit several candidate models (Parts 9, 10, 11, 20)
models = {
    "Linear": Pipeline([("prep", preprocessor), ("model", LinearRegression())]),
    "Ridge":  Pipeline([("prep", preprocessor), ("model", Ridge(alpha=1.0))]),
    "Lasso":  Pipeline([("prep", preprocessor), ("model", Lasso(alpha=1.0))]),
}

# 10. Train, predict, and evaluate every model on the SAME test set (Parts 15-17)
results = {}
for name, pipeline in models.items():
    pipeline.fit(X_train, y_train)
    preds = pipeline.predict(X_test)
    mse = mean_squared_error(y_test, preds)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, preds)
    results[name] = {"MSE": mse, "RMSE": rmse, "R2": r2}

results_df = pd.DataFrame(results).T
print(results_df)

# 11. Visualize the best model's fit and residuals (Part 27)
best_name = results_df["R2"].idxmax()
best_model = models[best_name]
best_preds = best_model.predict(X_test)

residuals = y_test - best_preds
plt.scatter(best_preds, residuals)
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Predicted price")
plt.ylabel("Residual")
plt.title(f"Residuals — {best_name} model")
plt.show()
```

How each step maps back to this guide

Step	Concept	Part
Load + inspect	`head()`, `info()`, `describe()`	3
Clean	`drop()`, missing values	3, 24
Explore	mean, variance, covariance, correlation	4
Split	train/test split, leakage-safe ordering	14, 23
Preprocess	imputing, scaling, encoding, `ColumnTransformer`	22, 24
Model	Linear / Multiple Linear / Polynomial / Ridge / Lasso	9–12, 20
Evaluate	MSE, RMSE, R^2, comparison table	15–17
Diagnose	under/overfitting, bias–variance	18–19
Visualize	scatter, fitted line/curve, residual plot	27

This template is intentionally reusable: swapping in a different Kaggle CSV, a different target column name, and a different list of numeric/categorical features adapts the entire pipeline to a new regression problem while preserving every good practice covered in this guide — a clean train/test split done first, leakage-free preprocessing, multiple candidate models evaluated fairly on the same held-out data, and visual diagnostics alongside numerical metrics.

Closing Notes

Every part of this guide builds on the one before it: raw data inspection feeds into statistical understanding, which feeds into the mechanics of least squares, which is implemented in scikit-learn, extended to multiple features and curved relationships, evaluated rigorously with a proper train/test split and multiple metrics, protected against overfitting with regularization, and finally assembled into one reusable, leakage-free workflow. The single habit worth carrying forward from this guide is to always ask, at each step: *what would happen if I ran this on data the model has never seen?* — that question is the thread connecting train/test splitting, cross-validation intuition, overfitting, regularization, and honest model evaluation.